

Linear Search.

In Linear Search, we iterate over all the elements of the array & check if the current element is equal to the target element.

- Best Case: $O(1)$
- Worst Case: $O(N)$, N is size of list
- Average Case: $O(N)$
- Auxiliary Case: $O(1)$

Solution.

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include <stdlib.h>
```

```
int linearSearch(int *arr, int N, int x) {
    int i = 0;
```

```
    for (i = 0; i < N; i++) {
```

```
        if (x == *(arr + i)) break;
```

```
    }
```

```
    if (i == N) printf("Not found");
```

```
    else printf("Found at index: %d", i);
```

```
}
```



```
int main() {  
    int i = 0;  
    int N, X = 0;
```

Top:

```
    printf("Enter number student: ");  
    scanf("%d", &N);  
    if (N <= 0) goto Top;
```

```
    int *roll-number = malloc (N * sizeof(int));  
    if (roll-number == NULL) {  
        fprintf(stderr, "memory allocation failed");  
        return 1;  
    }
```

```
    printf("Enter student Roll Number: ");  
    for (i = 0; i < N; i++) {  
        scanf("%d", (roll-number+i));  
        for (int j = 0; j < i; j++) {  
            if (*(roll-number+i) == *(roll-number+j)) {  
                printf("Roll Number Already exist");  
                i--;  
                break;  
            }  
        }  
    }
```

```
    printf("Enter roll to search: ");  
    scanf("%d", &X);  
    linearSearch (roll-number, N, X);
```

```
    free (roll-number);  
    return 0;  
}
```

```

C bubbleSort.c U C binarySearch.c U C linearSearch.c U X
C linearSearch.c > main()
1
2 #include <stdio.h>
3 #include <stdbool.h>
4 #include <stdlib.h>
5
6 void lineaeSearch( int *arr, int N , int X){
7     int i =0;
8     for( i =0 ; i < N ;){
9         if(X == *(arr+i) ) break;
10        i++;
11    }
12
13    if(i == N) printf("Not Found");
14    else printf("Found at index: %d",i);
15
16 }
17
18 int main(){
19     int i =0;
20     int N ;//Number Student
21     int X =0; // the roll number to be found
22
23     Top:
24     printf("Enter number student: ");
25     scanf("%d",&N);
26     if (N <= 0)
27         goto Top;
28
29     int *roll_number = malloc(N*sizeof(int)); // Array of N element to collect all student roll number
30     // checking malloc failure
31     if (roll_number == NULL){
32         fprintf(stderr,"Memory allocation failed\n");
33         return 1;
34     }
35     printf("Enter Students Roll Number: \n");
36     for (i =0 ; i<N ;i++){
37         scanf("%d",(roll_number+i));
38         for(int j =0; j<i;j++){
39             if (*(roll_number+i) == *(roll_number+j)){
40                 printf("Roll Number Already exists: Re-enter it.\n");
41                 i--;
42                 break;
43             }
44         }
45     }
46     printf("Enter roll to search: ");
47     scanf("%d",&X);
48
49     lineaeSearch(roll_number , N ,X);
50
51
52 free(roll_number);
53 return 0;
54 }

```

```

CHAT TERMINAL
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/
abhinavsharma/Comp. Language/C/" && gcc linearSearc
h.c -o linearSearch && "/Users/abhinavsharma/Comp. L
anguage/C/"linearSearch
Enter number student: 7
Enter Students Roll Number:
1
4
2
5
6
3
6
Roll Number Already exists: Re-enter it.
7
Enter roll to search: 4
Found at index: 1
abhinavsharma@Abhinavs-MacBook-Air C %

```


Binary Search:

Binary search is a searching algorithm that operates on a sorted or monotonic search space, repeatedly dividing it into halves to find a target value or optimal answer logarithmic time $O(\log N)$.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void bubbleSort(int* product_id, int N,  
                int target_id) {
```

```
    int mid, start = 0, end = N - 1;
```

```
    while (start <= end) {
```

```
        mid = start + (end - start) / 2;
```

```
        if (target_id == *(product_id + mid)) {  
            printf("Product available");  
            break;
```

```
        }
```

```
        else {
```

```
            if (target_id > *(product_id + mid)) {
```

```
                start = mid + 1;
```

```
            }
```

```
            else end = mid - 1;
```

```
        }
```


Date

```
if (Cend < Start) == 1) printf("Product  
Not Available");
```

```
}
```

```
int main() {
```

```
    int N, target_id, i;
```

```
Top:
```

```
    printf("Enter number of product:");
```

```
    scanf("%d", &N);
```

```
    if (N <= 0) goto Top;
```

```
    int *product_id = malloc(N * sizeof(int));
```

```
    if (product_id == NULL) {
```

```
        printf(stderr, "Memory allocation fail");
```

```
        return 1;
```

```
    }
```

```
    printf("Enter Product Id: \n");
```

```
    for (i = 0; i < N; i++) {
```

```
        scanf("%d", &product_id[i]);
```

```
    printf("Enter Product Id: \n");
```

```
    for (i = 0; i < N; i++) {
```

```
        scanf("%d", &product_id[i]);
```

```
    printf("Enter target product id: ");
```

```
    scanf("%d", &target_id);
```

```
    bubblesort(product_id, N, target_id);
```

```
    free(product_id);
```

```
    return 0;
```

```
}
```

C bubbleSort.c U

C binarySearch.c U X

C binarySearch.c > main()

1 /*An online store keeps product IDs sorted in ascending order. A customer wants to know whether a
2 particular product exists in inventory.*/
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 void bubbleSort(int *product_id ,int N,int target_id){
7 int mid,start =0, end =N-1;
8
9 while(start <= end){
10 mid = start + (end- start)/2;
11 if(target_id == *(product_id + mid)){
12 printf("Product Available");
13 break;
14 }
15 else{
16 if (target_id > *(product_id + mid)){
17 start = mid+1; // ignoring left half
18 }
19 else end = mid-1; // ignoring right half
20 }
21 }
22 if ((end<start) == 1) printf("Product Not Available");
23 }
24
25 int main(){
26 int N,target_id,i;
27
28 Top:
29 printf("Enter number of product: ");
30 scanf("%d",&N);
31 if (N<=0) goto Top;
32 int *product_id = malloc(N*sizeof(int)); // allocating memory to store product id
33 // check pointer is not NULL
34 if(product_id == NULL){
35 fprintf(stderr,"Memory allocation fail");
36 return 1;
37 }
38 printf("Enter Product Id: \n");
39 for(i =0 ; i<N ;i++){
40 scanf("%d",&product_id[i]);
41 }
42 printf("Enter target product id: ");
43 scanf("%d",&target_id);
44 bubbleSort(product_id , N , target_id);
45
46 free(product_id);
47 return 0;
48 }
49
50

CHAT

TERMINAL

abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/a
bhinavsharma/Comp. Language/C /" && gcc binarySearch
.c -o binarySearch && "/Users/abhinavsharma/Comp. La
nguage/C /"binarySearch
Enter number of product: 6
Enter Product Id:
1
2
4
5
7
9
Enter target product id: 7
Product Available
abhinavsharma@Abhinavs-MacBook-Air C %

Bubble Sort.

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent element if they are in the wrong order.

Time Complexity: $O(n^2)$

Auxiliary Space: $O(1)$.

Solution

```
#include <stdio.h>
```

```
void bubbleSort(int arr[], int n) {
```

```
    int i, j, temp;
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        for (j = 0; j < n - i - 1; j++) {
```

```
            if (arr[j] > arr[j + 1]) {
```

```
                temp = arr[j];
```

```
                arr[j] = arr[j + 1];
```

```
                arr[j + 1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```



```
int main () {
```

```
    int n, i;
```

```
    printf("Enter the number of students: ");
```

```
    scanf("%d", &n);
```

```
    int scores[n];
```

```
    printf("Enter the exam scores: ");
```

```
    for(i=0; i < n; i++) {
```

```
        scanf("%d", &scores[i]);
```

```
    }
```

```
    bubbleSort(scores, n);
```

```
    printf("Exam scores in ascending order: ");
```

```
    for(i=0; i < n; i++)
```

```
        printf("%d", scores[i]);
```

```
    }
```

```
    return 0;
```

```
}
```


C bubbleSort.c U X

C bubbleSort.c > main()

```
1  /*A teacher wants to arrange students' exam scores in ascending order to prepare the result sheet.
2  Write a program using Bubble Sort.*/
3  #include <stdio.h>
4
5  void bubbleSort(int arr[], int n) {
6      int i, j, temp;
7      for (i = 0; i < n-1; i++) {
8          for (j = 0; j < n-i-1; j++) {
9              if (arr[j] > arr[j+1]) {
10                 temp = arr[j];
11                 arr[j] = arr[j+1];
12                 arr[j+1] = temp;
13             }
14         }
15     }
16 }
17
18 int main() {
19     int n, i;
20     printf("Enter the number of students: ");
21     scanf("%d", &n);
22     int scores[n];
23     printf("Enter the exam scores: ");
24     for (i = 0; i < n; i++) {
25         scanf("%d", &scores[i]);
26     }
27     bubbleSort(scores, n);
28     printf("Exam scores in ascending order: ");
29     for (i = 0; i < n; i++) {
30         printf("%d ", scores[i]);
31     }
32     return 0;
33 }
```

CHAT TERMINAL

zsh + v [] | {} x

```
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/a
bhinavsharma/Comp. Language/C/" && gcc bubbleSort.c
-o bubbleSort && "/Users/abhinavsharma/Comp. Langua
ge/C/"bubbleSort
Enter the number of students: 5
Enter the exam scores: 54
63
85
23
76
Exam scores in ascending order: 23 54 63 76 85
abhinavsharma@Abhinavs-MacBook-Air C %
```

Selection Sort

Selection sort is a comparison-based sorting algorithm. It sorts by repeatedly selecting the smallest (or largest) element from the unsorted portion & swapping it with the first unsorted element.

Time complexity: $O(n^2)$

Auxiliary space: $O(1)$.

```
# include <stdio.h>
```

```
void selectionSort(int arr[], int n){  
    int i, j, min, temp;
```

```
    for (i = 0; i < n - 1; i++) {  
        min = i;
```

```
        for (j = i + 1; j < n; j++) {  
            if (arr[j] < arr[min]) {  
                min = j;
```

```
            }  
        }
```

```
        temp = arr[min];
```

```
        arr[min] = arr[i];
```

```
        arr[i] = temp;
```

```
    }
```

```
}
```



```
int main() {
```

```
    int arr[100], n, i;  
    printf("Enter the number of players: ");  
    scanf("%d", &n);  
    printf("Enter the scores of the players: \n");
```

```
    for (i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);
```

```
    }  
    selectionSort(arr, n);
```

```
    printf("Ranked scores of the player:");
```

```
    for (i = 0; i < n; i++) {  
        printf("%d ", arr[i]);
```

```
    }  
    return 0;
```

```
}
```

C selectionSort.c U X

C selectionSort.c > main()

```
1  /* A gaming club wants to rank players based on scores from lowest to highest.
2  Write a program using Selection Sort.*/
3  #include <stdio.h>
4
5  void selectionSort(int arr[], int n) {
6      int i, j, minIndex, temp;
7      for (i = 0; i < n - 1; i++) {
8          minIndex = i;
9          for (j = i + 1; j < n; j++) {
10             if (arr[j] < arr[minIndex]) {
11                 minIndex = j;
12             }
13         }
14         temp = arr[minIndex];
15         arr[minIndex] = arr[i];
16         arr[i] = temp;
17     }
18 }
19
20
21 int main() {
22     int arr[100], n, i;
23     printf("Enter the number of players: ");
24     scanf("%d", &n);
25     printf("Enter the scores of the players:\n");
26     for (i = 0; i < n; i++) {
27         scanf("%d", &arr[i]);
28     }
29     selectionSort(arr, n);
30     printf("Ranked scores of the players:\n");
31
32     for (i = 0; i < n; i++)
33     {
34         printf("%d ", arr[i]);
35     }
36
37     return 0;
38 }
```

CHAT TERMINAL

zsh + - - - | X

```
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/a
bhinavsharma/Comp. Language/C/" && gcc selectionSor
t.c -o selectionSort && "/Users/abhinavsharma/Comp.
Language/C/"selectionSort
Enter the number of players: 5
Enter the scores of the players:
```

```
4
6
2
8
1
Ranked scores of the players:
1 2 4 6 8
```

```
abhinavsharma@Abhinavs-MacBook-Air C %
```


Insertion Sort:

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list.

- Best Case : $O(n)$
- Average case : $O(n^2)$
- Worst case : $O(n^2)$

Solution:

```
#include <stdio.h>
```

```
void insertionSort(int arr[], int n){
```

```
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
```

```
        while (j >= 0 && arr[j] > key) {
```

```
            arr[j+1] = arr[j];
```

```
            j = j - 1;
```

```
            arr[j+1] = key;
```

```
        }
```

```
int main() {
```

```
    int arr[100], n, i;
```

```
    printf("Enter the number of books: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the serial number of the book: ");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    insertionSort(arr, n);
```

```
    printf("Sorted serial number of the books: \n");
```

```
    return
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("%d", arr[i]);
```

```
    }
```

```
    return 0;
```

```
}
```



```
C selectionSort.c U C insertionSort.c U X C bubbleSort.c U
C insertionSort.c > main()
1  /*A librarian receives book serial numbers one by one and wants to place each new book in the correct
2  position.
3  Write a program using Insertion Sort to arrange book numbers.*/
4  #include <stdio.h>
5  void InsertionSort(int arr[], int n) {
6      int i, key, j;
7      for (i = 1; i < n; i++) {
8          key = arr[i];
9          j = i - 1;
10
11         while (j >= 0 && arr[j] > key) {
12             arr[j + 1] = arr[j];
13             j = j - 1;
14         }
15         arr[j + 1] = key;
16     }
17 }
18
19 int main() {
20     int arr[100], n, i;
21     printf("Enter the number of books: ");
22     scanf("%d", &n);
23     printf("Enter the serial numbers of the books:\n");
24     for (i = 0; i < n; i++) {
25         scanf("%d", &arr[i]);
26     }
27     InsertionSort(arr, n);
28     printf("Sorted serial numbers of the books:\n");
29
30     for(i = 0 ; i<n ; i++){
31         printf("%d ",arr[i]);
32     }
33     return 0;
34 }
```

```
CHAT TERMINAL
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/a
bhinavsharma/Comp. Language/C/" && gcc insertionSor
t.c -o insertionSort && "/Users/abhinavsharma/Comp.
Language/C/"insertionSort
Enter the number of books: 6
Enter the serial numbers of the books:
7
4
5
2
8
3
Sorted serial numbers of the books:
2 3 4 5 7 8
abhinavsharma@Abhinavs-MacBook-Air C %
```